

Chapter 8. Patterns to Make the Most of LLMs

LLMs today have some major limitations, but that doesn't mean your dream LLM app is impossible to build. The experience that you design for users of your application needs to work around, and ideally with, the limitations.

[Chapter 5](#) touched on the key trade-off we face when building LLM apps: the trade-off between *agency* (the LLM's capacity to act autonomously) and *reliability* (the degree to which we can trust its outputs). Intuitively, any LLM application will be more useful to us if it takes more actions without our involvement, but if we let agency go too far, the application will inevitably do things we wish it hadn't.

[Figure 8-1](#) illustrates this trade-off.

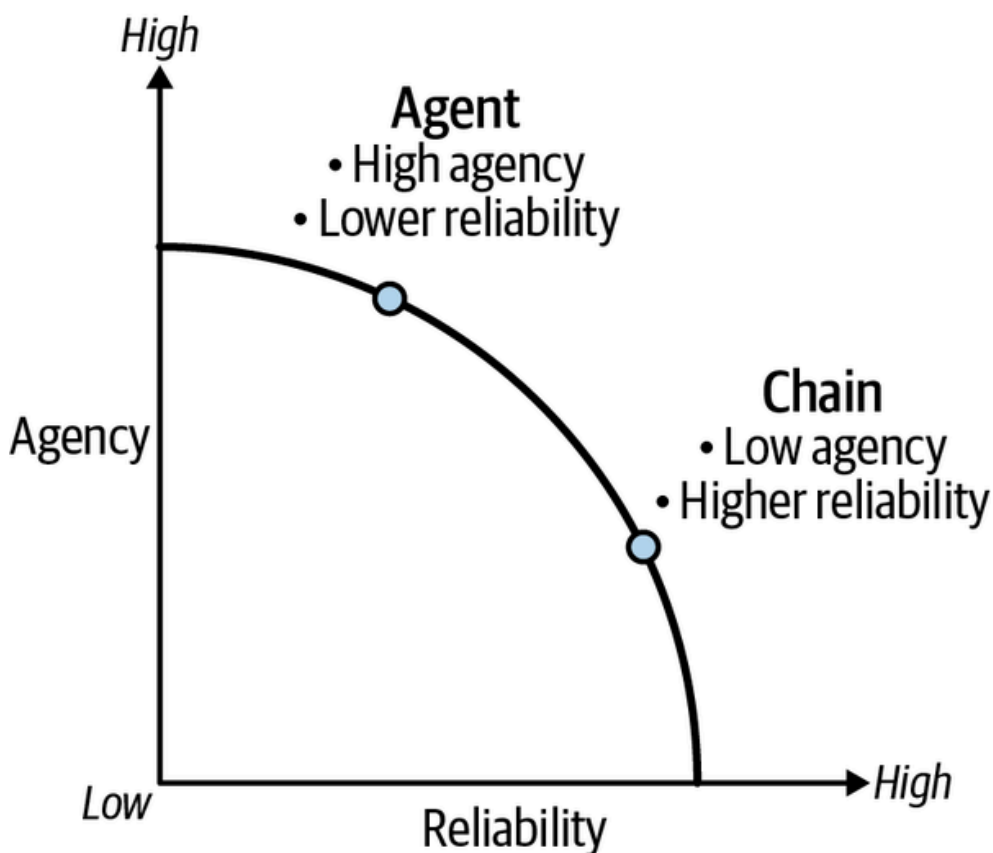


Figure 8-1. The agency-reliability trade-off

To borrow a concept from other fields,¹ we can visualize the trade-off as a *frontier*—all points on the frontier's curved line are optimal LLM architectures for some application, marking different choices between agency

and reliability. (Refer to [Chapter 5](#) for an overview of different LLM application architectures.) As an example, notice how the chain architecture has relatively low agency but higher reliability, whereas the Agent architecture has higher agency at the expense of lower reliability.

Let's briefly touch on a number of additional (but still important) objectives that you might want your LLM application to have. Each LLM app will be designed for a different mix of one or more of these objectives:

Latency

Minimize time to get final answer

Autonomy

Minimize interruptions for human input

Variance

Minimize variation between invocations

This is not meant as an exhaustive list of all possible objectives, but rather as illustrative of the trade-offs you face when building your application. Each objective is somewhat at odds with all the others (for instance, the easiest path to higher reliability requires either higher latency or lower autonomy). Each objective would nullify the others if given full weight (for instance, the minimal latency app is the one that does nothing at all). [Figure 8-2](#) illustrates this concept.

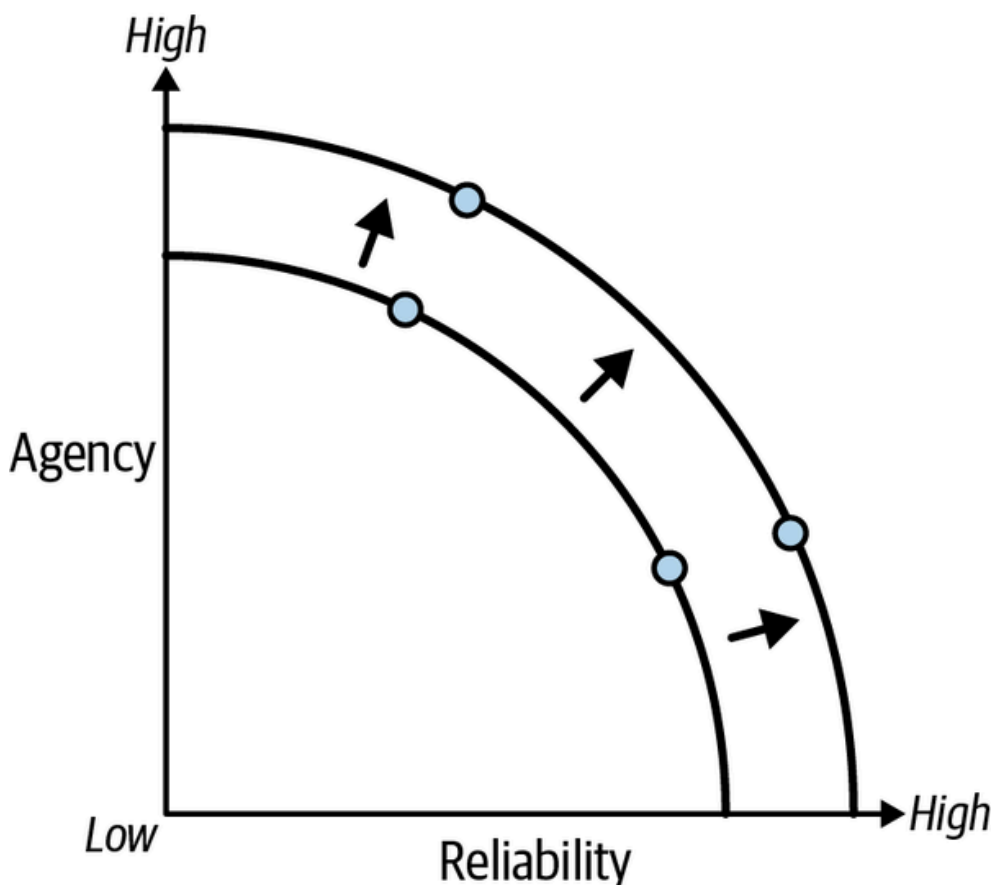


Figure 8-2. Shifting the frontier, or more agency, for the same reliability

What we really want as application developers is to shift the frontier outward. For the same level of reliability, we'd like to achieve higher agency; and for the same level of agency, we'd like to achieve higher reliability. This chapter covers a number of techniques you can use to achieve this:

Streaming/intermediate output

Higher latency is easier to accept if there is some communication of progress/intermediate output throughout.

Structured output

Requiring an LLM to produce output in a predefined format makes it more likely that it will conform to expectations.

Human in the loop

Higher-agency architectures benefit from human intervention while they're running: interrupting, approving, forking, or undoing.

Double texting modes

The longer an LLM app takes to answer, the more likely it is that the user might send it new input before the previous one has finished being processed.

Structured Output

It is often crucial to have LLMs return structured output, either because a downstream use of that output expects a things in a specific *schema* (a definition of the names and types of the various fields in a piece of structured output) or purely to reduce variance to what would otherwise be completely free-form text output.

There are a few different strategies you can use for this with different LLMs:

Prompting

This is when you ask the LLM (very nicely) to return output in the desired format (for instance, JSON, XML, or CSV). Prompting's big advantage is that it works to some extent with any LLM; the downside is that it acts more as a suggestion to the LLM and not as a guarantee that the output will come out in this format.

Tool calling

This is available for LLMs that have been fine-tuned to pick from a list of possible output schemas, and to produce something that con-

forms to the chosen one. This usually involves writing, for each of the possible output schemas: a name to identify it, a description to help the LLM decide when it is the appropriate choice, and a schema for the desired output format (usually in JSONSchema notation).

JSON mode

This is a mode available in some LLMs (such as recent OpenAI models) that enforces the LLM to output a valid JSON document.

Different models may support different variants of these, with slightly different parameters. To make it easy to get LLMs to return structured output, LangChain models implement a common interface, a method called `.with_structured_output`. By invoking this method—and passing in a JSON schema or a Pydantic (in Python) or Zod (in JS) model—the model will add whatever model parameters and output parsers are necessary to produce and return the structured output. When a particular model implements more than one of the preceding strategies, you can configure which method to use.

Let's create a schema to use:

Python

```
from pydantic import BaseModel, Field

class Joke(BaseModel):
    setup: str = Field(description="The setup of the joke")
    punchline: str = Field(description="The punchline to the joke")
```

JavaScript

```
import { z } from "zod";

const joke = z.object({
  setup: z.string().describe("The setup of the joke"),
  punchline: z.string().describe("The punchline to the joke"),
});
```

Notice how we take care to add a description to each field. This is key because—together with the name of the field—this is the information the LLM will use to decide what part of the output should go in each field. We could also have defined a schema in raw JSONSchema notation, which would look like this:

```
{'properties': {'setup': {'description': 'The setup of the joke',
    'title': 'Setup',
    'type': 'string'},
    'punchline': {'description': 'The punchline to the joke',
    'title': 'Punchline',
    'type': 'string'}},
    'required': ['setup', 'punchline'],
    'title': 'Joke',
    'type': 'object'}
```

And now let's get an LLM to generate output that conforms to this schema:

Python

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
model = model.with_structured_output(Joke)

model.invoke("Tell me a joke about cats")
```

JavaScript

```
import { ChatOpenAI } from "@langchain/openai";

let model = new ChatOpenAI({
  model: "gpt-3.5-turbo",
  temperature: 0
});
model = model.withStructuredOutput(joke);

await structuredLlm.invoke("Tell me a joke about cats");
```

An example of output

```
{
  setup: "Why don't cats play poker in the wild?",
  punchline: "Too many cheetahs."
}
```

A couple of things to notice:

- We create the instance of the model as usual, specifying the model name to use and other parameters.

- Low temperature is usually a good fit for structured output, as it reduces the chance the LLM will produce invalid output that doesn't conform to the schema.
- Afterward, we attach the schema to the model, which returns a new object, which will produce output that matches the schema provided. When you pass in a Pydantic or Zod object for schema, this will be used for validation as well; that is, if the LLM produces output that doesn't conform, a validation error will be returned to you instead of the failed output.
- Finally, we invoke the model with our (free-form) input, and receive back output that matches the structure we desired.

This pattern of using structured output can be very useful both as a standalone tool and as a part of a larger application; for instance, refer back to [Chapter 5](#), where we make use of this capability to implement the routing step of the router architecture.

Intermediate Output

The more complex your LLM architecture becomes, the more likely it will take longer to execute. If you think back to the architecture diagrams in Chapters [5](#) and [6](#), every time you see multiple steps (or nodes) connected in sequence or in a loop, that is an indication that the time it takes for a full invocation is increasing.

This increase in latency—if not addressed—can be a blocker to user adoption of LLM applications, with most users expecting computer applications to produce some output within seconds. There are several strategies to make the higher latency more palatable, but they all fall under the umbrella of *streaming output*, that is, receiving output from the application while it is still running.

For this section, we'll use the last architecture described in [“Dealing with Many Tools”](#). Refer back to [Chapter 6](#) for the full code snippet.

To generate intermediate output with LangGraph, all you have to do is to invoke the graph with the `stream` method, which will yield the output of each node as soon as each finishes. Let's see what that looks like:

Python

```
input = {
    "messages": [
        HumanMessage("""How old was the 30th president of the United States
when he died?""")
    ]
}
```

```
for c in graph.stream(input, stream_mode='updates'):
    print(c)
```

JavaScript

```
const input = {
  messages: [
    new HumanMessage(`How old was the 30th president of the United States
      he died?`)
  ]
}
const output = await graph.stream(input, streamMode: 'updates')
for await (const c of output) {
  console.log(c)
}
```

The output:

```
{
  "select_tools": {
    "selected_tools": ['duckduckgo_search', 'calculator']
  }
}
{
  "model": {
    "messages": AIMessage(
      content="",
      tool_calls=[
        {
          "name": "duckduckgo_search",
          "args": {
            "query": "30th president of the United States"
          },
          "id": "9ed4328dcdea4904b1b54487e343a373",
          "type": "tool_call",
        }
      ],
    )
  }
}
{
  "tools": {
    "messages": [
      ToolMessage(
        content="Calvin Coolidge (born July 4, 1872, Plymouth, Ver
          U.S.—died January 5, 1933, Northampton, Massachusetts)
          the 30th president of the United States (1923–29). Co
          acceded to the presidency after the death in office of
          Warren G. Harding, just as the Harding scandals were c
          to light....",
```

```

        name="duckduckgo_search",
        tool_call_id="9ed4328dcdea4904b1b54487e343a373",
    )
    ]
}
{
    "model": {
        "messages": AIMessage(
            content="Calvin Coolidge, the 30th president of the United Sta
            was born on July 4, 1872, and died on January 5, 1933. To
            calculate his age at the time of his death, we can subtrac
            birth year from his death year. \n\nAge at death = Death y
            Birth year\nAge at death = 1933 - 1872\nAge at death = 61
            years\n\nCalvin Coolidge was 61 years old when he died.",
        )
    }
}

```

Notice how each output entry is a dictionary with the name of the node that emitted as the key and the output of that node as the value. This gives you two key pieces of information:

- Where the application currently is; that is, if you think back to the architecture diagrams shown in previous chapters, where in that diagram are we currently?
- Each update to the shared state of the application, which together build up to the final output of the graph.

In addition, LangGraph supports more stream modes:

- `updates`. This is the default mode, described above.
- `values`. This mode yields the current state of the graph every time it changes, that is after each set of nodes finishes executing. This can be useful when the way you display output to your users closely tracks the shape of the graph state.
- `debug`. This mode yields detailed events every time something happens in your graph, including:
 - `checkpoint` events, whenever a new checkpoint of the current state is saved to the database
 - `task` events, emitted whenever a node is about to start running
 - `task_result` events, emitted whenever a node finishes running
- Finally, you can combine these modes; for instance, requesting both `updates` and `values` by passing a list.

You control the stream mode with the `stream_mode` argument to `stream()`.

Streaming LLM Output Token-by-Token

Sometimes you may also want to get streaming output from each LLM call inside your larger LLM application. This can be useful for various projects, such as when building an interactive chatbot, where you want each word to be displayed as soon as it is produced by the LLM.

You can achieve this with LangGraph as well:

Python

```
input = {
    "messages": [
        HumanMessage("""How old was the 30th president of the United States
                        when he died?""")
    ]
}
output = app.astream_events(input, version="v2")

async for event in output:
    if event["event"] == "on_chat_model_stream":
        content = event["data"]["chunk"].content
        if content:
            print(content)
```

JavaScript

```
const input = {
  messages: [
    new HumanMessage(`How old was the 30th president of the United States
                      he died?`)
  ]
}

const output = await agent.streamEvents(input, {version: "v2"});

for await (const { event, data } of output) {
  if (event === "on_chat_model_stream") {
    const msg = data.chunk as AIMessageChunk;
    if (msg.content) {
      console.log(msg.content);
    }
  }
}
```

This will emit each word (technically each token) as soon as it is received from the LLM. You can find more details on this pattern from [LangChain](#).

Human-in-the-Loop Modalities

As we walk the autonomy (or agency) ladder, we find ourselves increasingly giving up control (or oversight) in exchange for capability (or autonomy). The shared state pattern used in LangGraph (see [Chapter 5](#) for an introduction) makes it easier to observe, interrupt, and modify the application. This makes it possible to use many different *human-in-the-loop* modes, or ways for the developer/end user of an application to influence what the LLM is up to.

For this section, we'll again use the last architecture described in [“Dealing with Many Tools”](#). Refer back to [Chapter 6](#) for the full code snippet. For all human-in-the-loop modes, we first need to attach a checkpoint to the graph; refer to [“Adding Memory to StateGraph”](#) for more details on this:

Python

```
from langgraph.checkpoint.memory import MemorySaver

graph = builder.compile(checkpointer=MemorySaver())
```

JavaScript

```
import {MemorySaver} from '@langchain/langgraph'

graph = builder.compile({ checkpointer: new MemorySaver() })
```

This returns an instance of the graph that stores the state at the end of each step, so every invocation after the first doesn't start from a blank slate. Any time the graph is called, it starts by using the checkpoint to fetch the most recent saved state—if any—and combines the new input with the previous state. And only then does it execute the first nodes. This is key to enabling human-in-the-loop modalities, which all rely on the graph remembering the previous state.

The first mode, `interrupt`, is the simplest form of control—the user is looking at streaming output of the application as it is produced, and manually interrupts it when he sees fit (see [Figure 8-3](#)). The state is saved as of the last complete step prior to the user hitting the interrupt button. From there the user can choose to:

- Resume from that point onward, and the computation will proceed as if it hadn't been interrupted (see [“Resume”](#)).
- Send new input into the application (e.g., a new message in a chatbot), which will cancel any future steps that were pending and start dealing

with the new input (see “Restart”).

- Do nothing and nothing else will run.

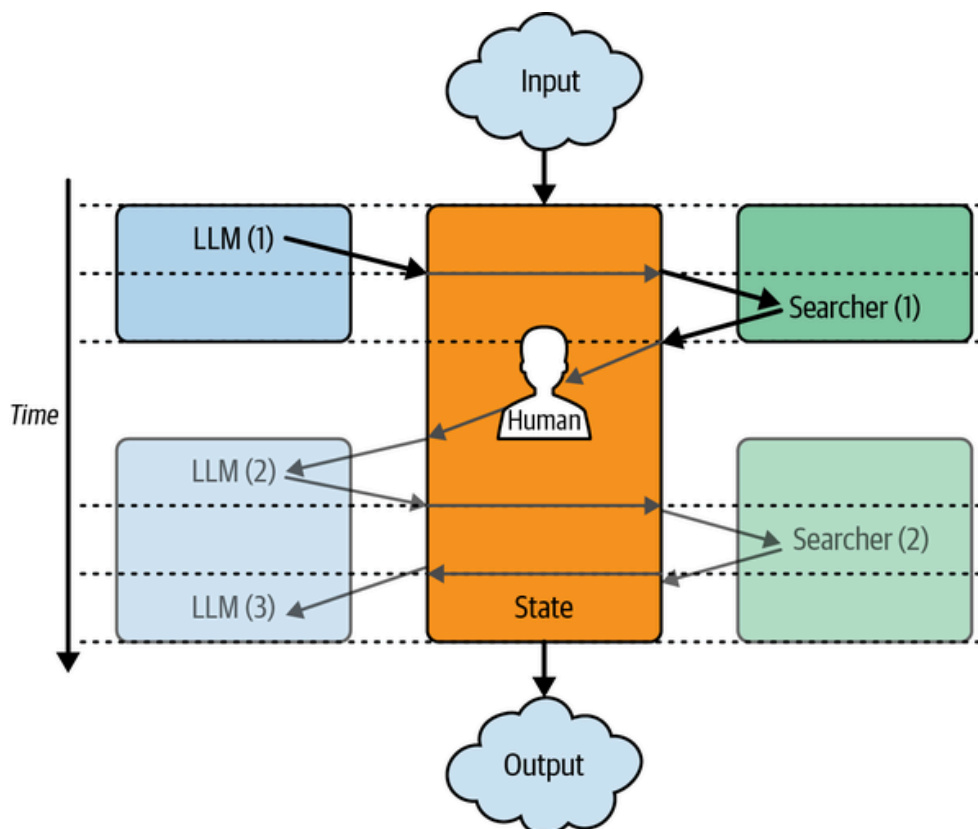


Figure 8-3. The `interrupt` pattern

Let's see how to do this in LangGraph:

Python

```
import asyncio

event = asyncio.Event()

input = {
    "messages": [
        HumanMessage("""How old was the 30th president of the United State
when he died?""")
    ]
}

config = {"configurable": {"thread_id": "1"}}

async with aclosing(graph.astream(input, config)) as stream:
    async for chunk in stream:
        if event.is_set():
            break
        else:
            ... # do something with the output

# Somewhere else in your application
```

```
event.set()
```

JavaScript

```
const controller = new AbortController()

const input = {
  "messages": [
    new HumanMessage(`How old was the 30th president of the United States
      he died?`)
  ]
}

const config = {"configurable": {"thread_id": "1"}}

try {
  const output = await graph.stream(input, {
    ...config,
    signal: controller.signal
  });
  for await (const chunk of output) {
    console.log(chunk); // do something with the output
  }
} catch (e) {
  console.log(e);
}

// Somewhere else in your application
controller.abort()
```

This makes use of an event or signal, so that you can control interruption from outside of the running application. Notice in the Python code block the use of `aclosing`; this ensures the stream is properly closed when interrupted. Notice in JS the use of the `try-catch` statement, as interrupting the run will result in an `abort` exception being raised. Finally notice that usage of the checkpointer requires passing in an identifier for this thread, to distinguish this interaction with the graph from all others.

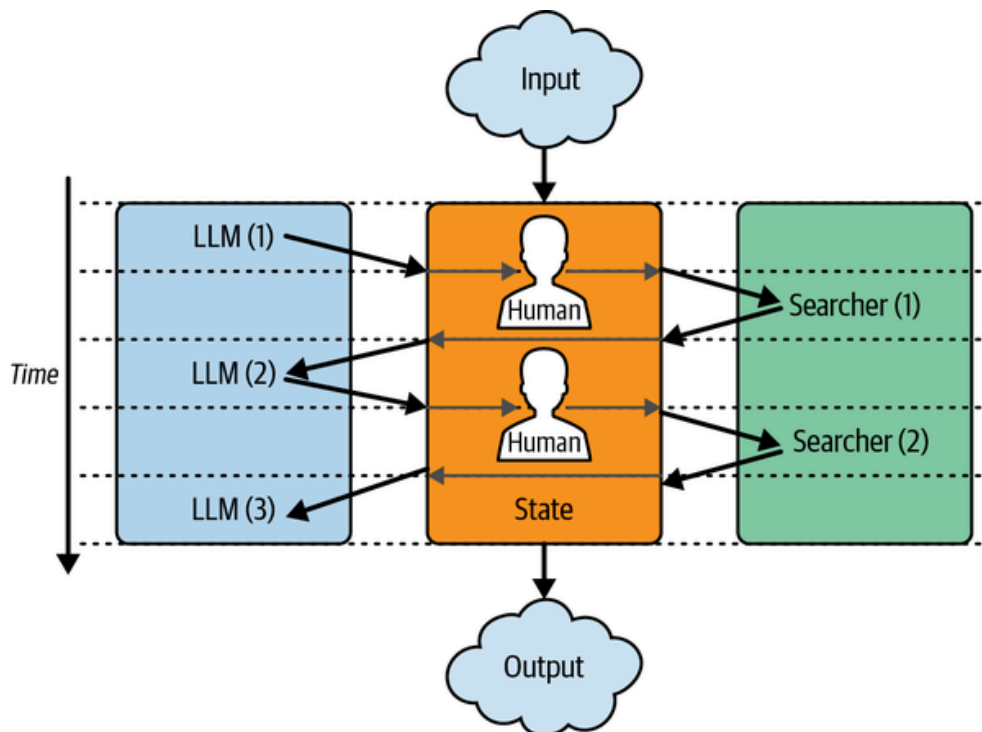


Figure 8-4. The `authorize` pattern

A second control mode is `authorize`, where the user defines ahead of time that they want the application to hand off control to them every time a particular node is about to be called (see [Figure 8-4](#)). This is usually implemented for tool confirmation—before any tool (or particular tools) is called, the application will pause and ask for confirmation, at which point the user can, again:

- Resume computation, accepting the tool call.
- Send a new message to guide the bot in a different direction, in which case the tool will not be called.
- Do nothing.

Here's the code:

Python

```
input = {
    "messages": [
        HumanMessage("""How old was the 30th president of the United States
when he died?""")
    ]
}

config = {"configurable": {"thread_id": "1"}}

output = graph.astream(input, config, interrupt_before=['tools'])

async for c in output:
    ... # do something with the output
```

```

const input = {
  "messages": [
    new HumanMessage(`How old was the 30th president of the United States
      he died?`)
  ]
}

const config = {"configurable": {"thread_id": "1"}}

const output = await graph.stream(input, {
  ...config,
  interruptBefore: ['tools']
});
for await (const chunk of output) {
  console.log(chunk); // do something with the output
}

```

This will run the graph up until it is about to enter the node called `tools`, thus giving you the chance to inspect the current state, and decide whether to proceed or not. Notice that `interrupt_before` is a list where order is not important; if you pass multiple node names, it will interrupt before entering each of them.

Resume

To proceed from an interrupted graph—such as when using one of the previous two patterns—you just need to re-invoke the graph with null input (or `None` in Python). This is taken as a signal to continue processing the previous non-null input:

Python

```

config = {"configurable": {"thread_id": "1"}}

output = graph.astream(None, config, interrupt_before=['tools'])

async for c in output:
    ... # do something with the output

```

JavaScript

```

const config = {"configurable": {"thread_id": "1"}}

const output = await graph.stream(null, {
  ...config,
  interruptBefore: ['tools']
}

```

```
});
for await (const chunk of output) {
  console.log(chunk); // do something with the output
}
```

Restart

If instead you want an interrupted graph to start over from the first node, with additional new input, you just need to invoke it with new input:

Python

```
input = {
    "messages": [
        HumanMessage("""How old was the 30th president of the United State
                        when he died?""")
    ]
}

config = {"configurable": {"thread_id": "1"}}

output = graph.astream(input, config)

async for c in output:
    ... # do something with the output
```

JavaScript

```
const input = {
  "messages": [
    new HumanMessage(`How old was the 30th president of the United States
                      he died?`)
  ]
}

const config = {"configurable": {"thread_id": "1"}}

const output = await graph.stream(input, config);

for await (const chunk of output) {
  console.log(chunk); // do something with the output
}
```

This will keep the current state of the graph, merge it with the new input, and start again from the first node.

If you want to lose the current state, just change the `thread_id`, which will start a new interaction from a blank slate. Any string value is a valid

`thread_id`; we'd recommend using UUIDs (or other unique identifiers) as thread IDs.

Edit state

Sometimes you might want to update the state of the graph before resuming; this is possible with the `update_state` method. You'll usually want to first inspect the current state with `get_state`.

Here's what it looks like:

Python

```
config = {"configurable": {"thread_id": "1"}}

state = graph.get_state(config)

# something you want to add or replace
update = { }

graph.update_state(config, update)
```

JavaScript

```
const config = "configurable": {"thread_id": "1"}

const state = await graph.getState(config)

// something you want to add or replace
const update = { }

await graph.updateState(config, update)
```

This will create a new checkpoint containing your update. After this, you're ready to resume the graph from this new point. See [“Resume”](#) to find out how.

Fork

You can also browse the history of all past states the graph has passed through, and any of them can be visited again, for instance, to get an alternative answer. This can be very useful in more creative applications, where each run through the graph is expected to produce different output.

Let's see what it looks like:


```
config = {"configurable": {"thread_id": "1"}}

history = [
    state for state in
    graph.get_state_history(config)
]

# replay a past state
graph.invoke(None, history[2].config)
```

JavaScript

```
const config = "configurable": {"thread_id": "1"}

const history = await Array.fromAsync(graph.getStateHistory(config))

// replay a past state
await graph.invoke(null, history[2].config)
```

Notice how we collect the history into a list/array in both languages; `get_state_history` returns an iterator of states (to allow consuming lazily). The states returned from the history method are sorted with the most recent first and the oldest last.

The true power of the human-in-the-loop controls comes from mixing them in whatever way suits your application.

Multitasking LLMs

This section covers the problem of handling concurrent input for LLM applications. This is a particularly relevant problem given that LLMs are quite slow, much more so when producing long outputs or when chained in multistep architectures (like you can do with LangGraph). Even as LLMs become faster, dealing with concurrent inputs will continue to be a challenge, as latency improvements will also unlock the door for more and more complex use cases, in much the same way as even the most productive person still faces the need to prioritize competing demands on their time.

Let's walk through the options.

Refuse concurrent inputs

Any input received while processing a previous one is rejected. This is the simplest strategy, but unlikely to cover all needs, as it effectively means handing off concurrency management to the caller.

Handle independently

Another simple option is to treat any new input as an independent invocation, creating a new thread (a container for remembering state) and producing output in that context. This has the obvious downside of needing to be shown to the user as two separate and unreconcilable invocations, which isn't always possible or desirable. On the other hand, it has the upside of scaling to arbitrarily large sizes, and is something you'll use to some extent in your application almost certainly. For instance, this is how you would think about the problem of getting a chatbot to "chat" with two different users concurrently.

Queue concurrent inputs

Any input received while processing a previous one is queued up and handled when the current one is finished. This strategy has some pros:

- It supports receiving an arbitrary number of concurrent requests.
- Because we wait for current input to finish processing, it doesn't matter if the new input arrives almost immediately after we start handling the current input or immediately before we finish; the end result will be the same, as we will finish processing the current input before moving on to the next.

The strategy suffers from a few drawbacks as well:

- It may take a while to process all queued inputs; in fact, the queue may grow unbounded if inputs are produced at a rate faster than processed.
- The inputs may be stale by the time they get processed, given that they are queued before seeing the response to the previous one, and not altered afterwards. This strategy is not appropriate when new inputs depend on previous answers.

Interrupt

When a new input is received while another is being processed, abandon processing of the current one and restart the chain with the new input. This strategy can vary by what is kept of the interrupted run. Here are a few options:

- Keep nothing. The previous input is completely forgotten, as if it had never been sent or processed.
- Keep the last completed step. In a checkpointing app (which stores progress as it moves through the computation), keep the state produced by the last completed step, discard any pending state updates from the currently executing step, and start handling the new input in that context.
- Keep the last completed step, as well as the current in-progress step. Attempt to interrupt the current step while taking care to save any incomplete updates to state that were being produced at the time. This is likely to not generalize beyond the simplest architectures.
- Wait for the current node (but not any subsequent nodes) to finish, then save and interrupt.

This option has some pros compared to queuing concurrent inputs:

- New input is handled as soon as possible, reducing latency and the chance of producing stale outputs.
- For the “keep nothing” variant, the final output doesn’t depend on when the new input was received.

But it also has drawbacks:

- Effectively, this strategy is still limited to processing one input at a time; any old input is abandoned when new input is received.
- Keeping partial state updates for the next run requires the state to be designed with that in mind; if not, then your application is likely to end up in an invalid state. For instance, OpenAI chat models require an AI message requesting tool calls to be immediately followed by tool messages with the tool outputs. If your run is interrupted in between, you either defensively clean up the intermediate state or risk being unable to progress further.
- The final outputs produced are very sensitive to when the new input is received; new input will be handled in the context of the (incomplete) progress previously made toward handling the previous input. This can result in brittle or unpredictable outcomes if you don’t design accordingly.

Fork and merge

Another option is to handle new input in parallel, forking the state of the thread as it is when the new input is received and merging the final states as inputs finish being handled. This option requires designing your state to either be mergeable without conflicts (e.g., using conflict-free replicated data types [CRDTs] or other conflict resolution algorithms) or having the user manually resolve conflicts before you’re able to make sense of

the output or send new input in this thread. If either of those two requirements is met, this is likely to be the best option overall. This way, new input is handled in a timely manner, output is independent of time received, and it supports an arbitrary number of concurrent runs.

Some of these strategies are implemented in LangGraph Platform, which will be covered in [Chapter 9](#).

Summary

In this chapter, we returned to the main trade-off you face when building LLM applications: agency versus reliability. We learned that there are strategies to partially beat the odds and get more reliability without sacrificing agency, and vice versa.

We started by covering structured outputs, which can improve the predictability of LLM-generated text. Next, we discussed emitting streaming/intermediate output from your application, which can make high latency (an inevitable side effect of agency currently) applications pleasant to use.

We also walked through a variety of human-in-the-loop controls—that is, techniques to give back some oversight to the end user of your LLM application—which can often make the difference in making high-agency architectures reliable. Finally, we talked about the problem of handling concurrent input to your application, a particularly salient problem for LLM apps given their high latency.

In the next chapter, you'll learn how to deploy your AI application into production.

- ¹ In finance, the *efficient frontier* in portfolio optimization; in economics, a *production-possibility frontier*; in engineering, the *Pareto front*.